

HCLS Skill Development Playbook

How to Create & Update Industry Skills
in 5 Minutes

Health Sciences Industry Solutions

March 2026

What This Playbook Covers

This playbook gives you the fastest path to creating or updating an HCLS industry skill for Cortex Code. It distills the full HCLS_INDUSTRY_SKILL_BEST_PRACTICES.md into actionable steps you can follow in minutes.

Two Tracks

Track A: Create a New Skill	Track B: Update an Existing Skill
5 steps to go from zero to a working skill in the incubator	4 steps to safely modify, test, and push an update

Prerequisites

- Git access to Snowflake-Solutions/health-sciences-coco-skills-incubator
- Cortex Code installed (cortex CLI)
- Snowflake account with ACCOUNTADMIN or appropriate role
- Familiarity with your domain (the skill teaches Cortex Code what YOU know)

Choose Your Archetype

Before starting, identify which pattern fits your skill:

Archetype	When to Use	Examples
Standalone	Single coherent workflow	FHIR, Pharmacovigilance, Survival Analysis, Variant Annotation, OMOP
Router + Sub-Skills	Multiple distinct workflows sharing a data model	Imaging (7 sub-skills), Clinical NLP (15 sub-skills)
CKE	Wrapping a Marketplace shared Cortex Search	PubMed CKE, ClinicalTrials.gov CKE
Compute-Heavy	External compute (containers, pipelines)	Nextflow nf-core, scvi-tools, Imaging ML

TIP: Most new skills are Standalone. Start there unless you have 3+ distinct workflows.

Track A: Create a New Skill

Step 1: Name & Scaffold

Estimated time: 30 seconds

Pick your skill name using the HCLS naming convention:

```
hcls-{segment}-{domain}-{capability}
```

Segments: provider | pharma | payer | meddev | cross

Domains: cdata | imaging | genomics | dsafety | lab |
claims | risk | quality | regulatory |
postmarket | engineering | cke

Create the skill directory:

```
cd health-sciences-coco-skills-incubator
mkdir -p skills/hcls-{your-name}
touch skills/hcls-{your-name}/SKILL.md
```

TIP: Use `hcls-cross-skill-development` skill in Cortex Code to auto-scaffold!

Step 2: Write Your SKILL.md

Estimated time: 2-3 minutes

Every SKILL.md has three required parts:

Part A: Frontmatter (REQUIRED)

```
---
name: hcls-{segment}-{domain}-{capability}
description: "What this skill does. Triggers: keyword1,
  keyword2, keyword3."
platform_affinities:
  produces: [tables, views]
  benefits_from:
    - skill: data-governance
      when: "output contains PHI"
---
```

Part B: Workflow (the core of your skill)

Write the step-by-step instructions that teach Cortex Code how to solve the domain problem. Include:

- Setup / prerequisites
- Step-by-step workflow with code examples (SQL, Python)
- Decision points where user input is needed
- Stopping points marked with: ****MANDATORY STOPPING POINT****

Part C: Output Section (REQUIRED)

Output

Describe what the skill produces: tables created, files generated, pipelines deployed, etc.

IMPORTANT: Keep SKILL.md under 500 lines. Only include what Cortex Code doesn't already know.

Step 3: Add Domain Knowledge

Estimated time: 1 minute

Your skill's value is the domain knowledge Cortex Code can't infer. Add what's needed for YOUR domain:

What You Need	How to Add It
Reference docs	Add to references/ directory (markdown, CSV) Loaded on-demand by the skill
SQL scripts	Add to scripts/ directory Setup scripts, DDL, stored procedures
Python helpers	Add to scripts/ directory Data processing, API calls, pipeline tools
Seed / reference data	Add to seed-data/ directory CSVs + setup SQL for COPY INTO
Domain PDFs	Add to skill root directory Taxonomy docs, schema docs, standards
Data model reference	Create data-model-knowledge/ sub-skill (for router skills with complex schemas)

TIP: Not every skill needs all of these. A simple standalone skill may only need SKILL.md + a SQL script.

Step 4: Register & Test

Estimated time: 1 minute

Register your skill in the incubator registry:

```
# Edit templates/skills_incubator.yaml
# Add your skill entry under the appropriate segment

# Regenerate the orchestrator
python scripts/generate_orchestrators.py --profile incubator
```

Test your skill locally:

```
# Option 1: Test via incubator profile
cortex --profile health-sciences-incubator
> "your trigger phrase here"

# Option 2: Test skill directly
cortex skill add ./skills/hcls-{your-name}
```

IMPORTANT: Always test your skill on real data before declaring it done. Verify outputs match expectations.

Step 5: Validate & Ship

Estimated time: 30 seconds

Run through the validation checklist, then commit:

Quick Validation Checklist

- ☐ SKILL.md has name + description in frontmatter
- ☐ platform_affinities declared (produces + benefits_from)
- ☐ At least one stopping point with MANDATORY STOPPING POINT marker
- ☐ ## Output section present
- ☐ Under 500 lines
- ☐ Tested on representative data
- ☐ Registered in skills_incubator.yaml
- ☐ Orchestrator regenerated

Create skill_evidence.yaml:

```
skill_name: "hcls-{your-name}"
skill_type: "customer-facing"
promotion_stage: "draft"
version: "1.0.0"
authors:
  - "sfc-gh-{your-ldap}"
```

Commit and push:

```
git checkout -b feature/hcls-{your-name}
git add skills/hcls-{your-name}/
git add templates/skills_incubator.yaml
git add agents/health-sciences-incubator.md
git commit -m "Add hcls-{your-name} skill"
git push -u origin feature/hcls-{your-name}
# Open PR to main
```

Track B: Update an Existing Skill

Step 1: Branch & Understand

Estimated time: 30 seconds

```
git checkout -b fix/hcls-{skill-name}-{description}

# Read the current skill
cat skills/hcls-{skill-name}/SKILL.md

# Understand the intent table (for router skills)
grep -A 20 'Intent Detection' skills/hcls-{skill-name}/SKILL.md
```

Step 2: Make Your Changes

Estimated time: 2-3 minutes

Common update patterns:

Update Type	What to Change
Add a workflow step	Add step to SKILL.md workflow. Include SQL/Python examples. Add stopping point if step needs approval.
Add a sub-skill (router only)	Create new directory + SKILL.md inside parent skill. Add parent_skill to frontmatter. Add intent to router table.
Fix a bug	Edit the specific step/code. Update any affected references or cross-checks.
Add domain knowledge	Add files to references/ or seed-data/. Update SKILL.md to reference new files.
Update platform affinities	Edit frontmatter produces/benefits_from. Regenerate orchestrator if affinities changed.
Add governance	Create governance/ sub-skill or add governance section to SKILL.md. Tag PHI columns.

Step 3: Test Your Changes

Estimated time: 1 minute

- For workflow changes: run the modified steps on sample data
- For sub-skill additions: test intent routing + sub-skill execution
- For bug fixes: reproduce the bug first, then verify the fix
- For domain knowledge: verify references load correctly in the workflow

IMPORTANT: If you modified a router's intent table, test ALL intents (not just the one you changed).

Step 4: Validate & Push

Estimated time: 30 seconds

Run the quick validation checklist from Track A Step 5, then:

```
git add -A && git commit -m "Update hcls-{skill-name}: {description}"  
git push -u origin fix/hcls-{skill-name}-{description}  
# Open PR to main
```


Quick Reference: Router Skill Anatomy

If you're building a router skill (Archetype B), here's the structure at a glance:

```
skills/hcls-{name}/
  SKILL.md                      # Router (intent table + preflight + Step 0)
  skill_evidence.yaml           # Promotion lifecycle tracking
  data-model-knowledge/         # Schema grounding sub-skill
    SKILL.md
    references/model.csv
    scripts/setup.sql
    seed-data/
  {sub-skill-a}/SKILL.md        # Functional sub-skills
  {sub-skill-b}/SKILL.md
  governance/SKILL.md           # If handling PHI/sensitive data
  pipeline-implementation/      # If production pipeline needed
    SKILL.md
```

Router SKILL.md Template

```
---
name: hcls-{name}
description: "..."
platform_affinities:
  produces: [...]
  benefits_from: [...]
---

# Title

## Preflight Check
(probe for data model knowledge service)

## Intent Detection
| Intent | Triggers | Load |
| INTENT_A | keywords... | sub-skill-a/SKILL.md |
| INTENT_B | keywords... | sub-skill-b/SKILL.md |

## Step 0: Data Model Knowledge (conditional)
(query Cortex Search, pass context to sub-skill)

## Stopping Points
- MANDATORY STOPPING POINT: after intent if ambiguous
- MANDATORY STOPPING POINT: before creating DB objects

## Output
What the skill produces overall.
```

Sub-Skill SKILL.md Template

```
---
name: {sub-skill-name}
description: "...
parent_skill: hcls-{parent-name}
---

# Title

## Workflow
Step 1: ...
Step 2: ...

## Stopping Points
- MANDATORY STOPPING POINT: ...

## Output
What this sub-skill produces.
```

Quick Reference: Governance Patterns

Choose the right governance approach based on your data sensitivity:

Data Type	Governance Level	What to Implement
PHI (patient names, MRNs, dates, notes)	High (dedicated sub-skill)	Tag-based PHI classification, column masking, row-access policies, 3-tier roles, AI guardrails, ML de-identification, audit queries
Patient-level (claims, adverse events, encounters)	Medium (section in SKILL.md)	Column masking on identifiers, role-based access, platform affinity to data-governance
Identifiable genomic data (sample-to-patient)	Medium (section in SKILL.md)	De-identification of mapping tables, secure compute for variant data, access policies
Aggregate / de-identified data only	Low (affinity only)	Declare platform affinity to data-governance. No masking needed for aggregate data.
Reference data (CKEs, ontologies, public datasets)	None	No governance needed. Read-only shared services with no patient data.

Quick Reference: Cross-Validation Audit

Before promoting a skill, audit it against the best practices. Use this severity guide:

Sev.	Check	What to Verify
CRIT	Frontmatter	name + description present, trigger keywords included
CRIT	Workflow	At least one stopping point, no chaining without approval
CRIT	Output	## Output section present in every skill and sub-skill
WARN	File size	SKILL.md under 500 lines (router can be slightly longer)
WARN	Naming	Follows hcls-{segment}-{domain}-{capability} convention
WARN	Affinities	platform_affinities declared with produces + benefits_from
WARN	Sub-skills	parent_skill declared in all sub-skill frontmatter
WARN	Evidence	skill_evidence.yaml with promotion_stage
PASS	Preflight	External dependencies have READY/MISSING/ERROR handling
PASS	Testing	Domain-specific validation tests documented/passed

Cheat Sheet: Common Commands

Skill Development

```
# Scaffold a new skill (via Cortex Code)
cortex> "create a new HCLS skill for {domain}"

# Test skill locally
cortex skill add ./skills/hcls-{name}

# Test via incubator profile
cortex --profile health-sciences-incubator
```

Orchestrator Management

```
# Regenerate orchestrators after registry changes
python scripts/generate_orchestrators.py --profile both

# Validate orchestrator integrity
python scripts/qa_validate_orchestrator.py
```

Git Workflow

```
# New skill
git checkout -b feature/hcls-{name}

# Bug fix
git checkout -b fix/hcls-{name}-{desc}

# Documentation
git checkout -b docs/{desc}

# Push and PR
git push -u origin {branch}
gh pr create --title '{title}' --body '{description}'
```

Useful Queries

```
# Count all skills
ls -d skills/hcls-*/ | wc -l

# Check all SKILL.md line counts
wc -l skills/*/SKILL.md | sort -rn

# Verify all frontmatter has name field
grep -l '^name:' skills/*/SKILL.md | wc -l

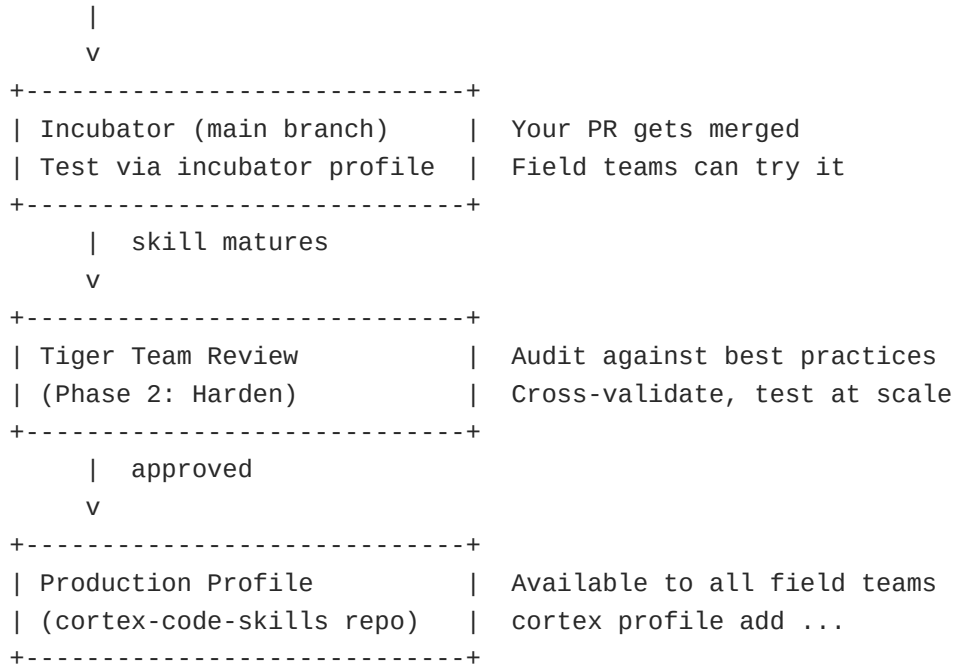
# Check all have ## Output sections
```

HCLS Skill Development Playbook
grep -rl '^## Output' skills/*/SKILL.md | wc -l

What's Next?

You've created or updated a skill. Here's what happens next in the lifecycle:

YOU ARE HERE



Key Resources

- HCLS_INDUSTRY_SKILL_BEST_PRACTICES.md - Full best practices guide (repo root)
- SKILL_BEST_PRACTICES.md - Platform skill best practices (cortex-code-skills repo)
- documentation/ISF-*.pdf - Industry Solutions Framework lifecycle
- templates/skills_incubator.yaml - Skill registry
- agents/health-sciences-incubator.md - Orchestrator (auto-generated)

Getting Help

- Use hcls-cross-skill-development skill in Cortex Code for guided scaffolding
- File an issue on the incubator repo for questions or feature requests
- Tag the Tiger Team for promotion reviews